

Strukture podataka i algoritmi

Algoritmi sortiranja 2

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Sortiranje spajanjem (Merge sort)
2. Quick Sort algoritam sortiranja
3. Radix Sort

Sortiranje spajanjem (MergeSort)

Sortiranje spajanjem (Merge sort)

- Algoritmi sa vremenskom složnošću $O(n^2)$ nisu prikladni za sortiranje velikih nizova, pa tada možemo koristiti brže algoritme koji koriste strategiju "podeli pa vladaj"
- Njihova prosečna vremenska složenost je $O(n \log_2 n)$ a **MergeSort** njihov tipični predstavnik
- Ulazni niz se sortira rekurzivno tako što se podeli na dva jednaka dela koja se prvo sortiraju pa se potom vrši spajanje njihove dve sortirane polovine
- U svakom rekurzivnom pozivu prosleđeni podniz deli se na polovine koje se sortiraju i potom spajaju
- Osnovni slučaj rekurzije nastupa kada niz koji treba sortirati sadrži samo jedan element - takva niz je već sortiran

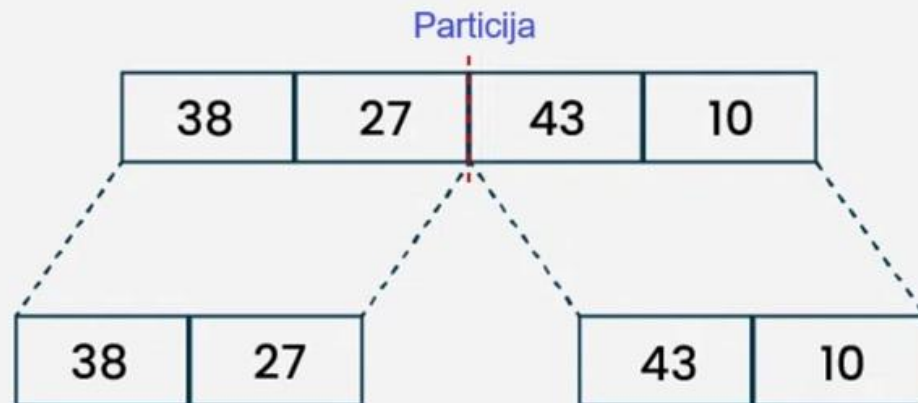
MergeSort

- Koraci 1 i 2: **Podela**

- [38, 27, 43, 10] niz je podeljen na podnizove [38, 27] i [43, 10]
- [38, 27] je podeljen na [38] i [27]
- [43, 10] je podeljen na [43] i [10]

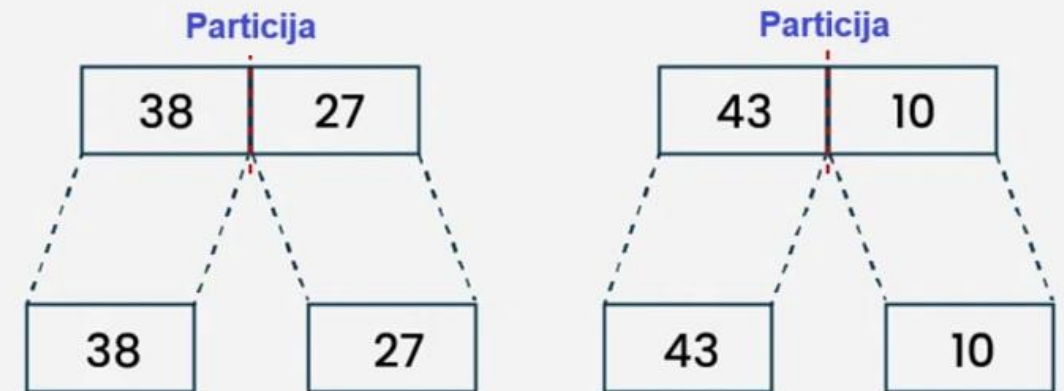
KORAK 1

Podeliti niz na dve jednake polovine



KORAK 2

Podeliti niz na dve jednake polovine



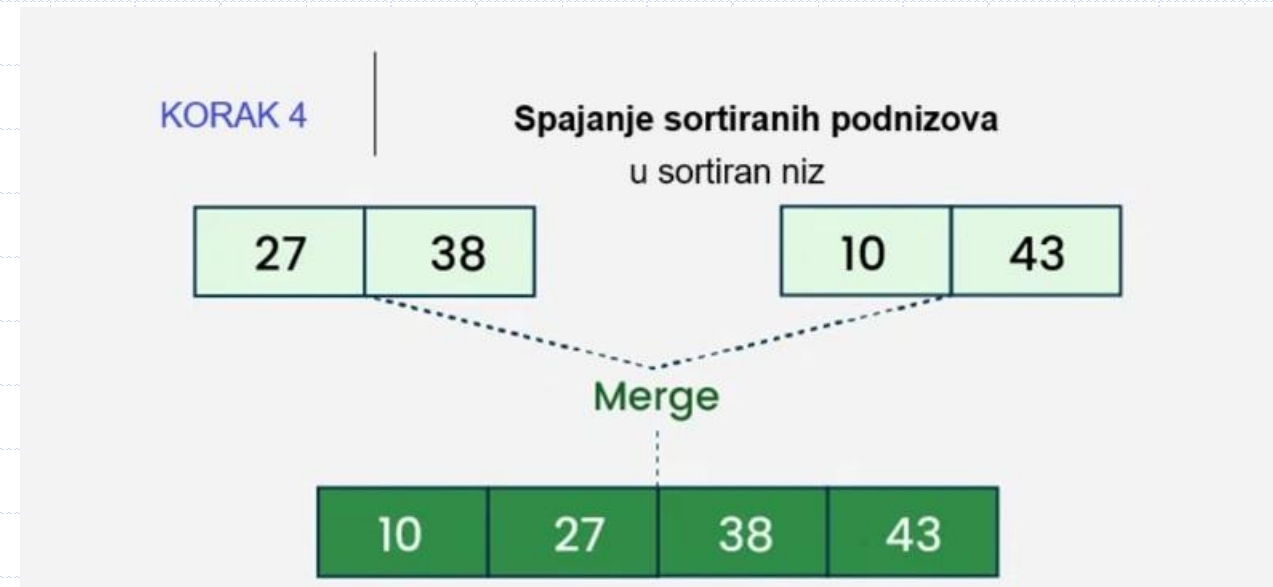
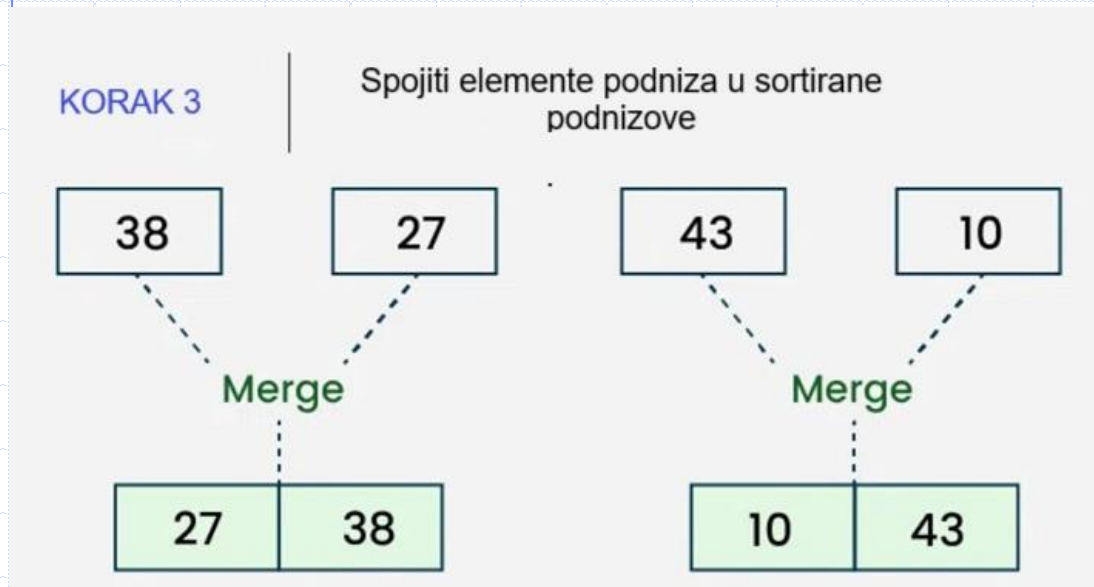
MergeSort

- Korak 3: Conquer

- [38] je već sortiran
- [27] je već sortiran
- [43] je već sortiran
- [10] je već sortiran

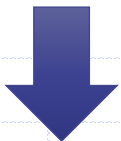
- Korak 4: Merge

- Spajanje [38] i [27] u niz [27, 38]
- Spajanje [43] i [10] u niz [10, 43]
- Spajanje [27, 38] i [10, 43] za dobijanje finalnog sortiranog niza [10, 27, 38, 43]



MergeSort

```
1 def merge_sort(array):
2     if len(array) <= 1:
3         return array
4     mid = len(array) // 2
5     left_arr = array[:mid]
6     right_arr = array[mid:]
7
8     return merge_sorted_arrays(merge_sort(left_arr), merge_sort(right_arr))
9
10 def merge_sorted_arrays(left_arr, right_arr):
11     sorted_arr = []
12     i = j = 0
13     while i < len(left_arr) and j < len(right_arr):
14         if left_arr[i] <= right_arr[j]:
15             sorted_arr.append(left_arr[i])
16             i += 1
17         else:
18             sorted_arr.append(right_arr[j])
19             j += 1
20
21     sorted_arr.extend(left_arr[i:])
22     sorted_arr.extend(right_arr[j:])
23
24     return sorted_arr
25
26 arr = [123, 87, 66, 49, 37, 25, 13, 1]
27 print(f"Ulazni niz: arr = {arr}")
28 print("Sortirani niz:", merge_sort(arr))
--
```



Ulazni niz: arr = [123, 87, 66, 49, 37, 25, 13, 1]
Sortirani niz: [1, 13, 25, 37, 49, 66, 87, 123]

Merge Sort algoritam je tipa „podeli pa vladaj“ koji rekursivno deli ulazni niz na dve polovine, sortira ih pojedinačno, a zatim ih ponovo spaja u jedan sortirani niz

- Funkcija `merge_sort` se rekursivno poziva na levoj i desnoj polovini ulaznog niza dok se ne dostigne osnovni slučaj (kada niz ima samo jedan element).
- Niz se deli na dve polovine pronalaženjem srednjeg indeksa (`mid`), i kreiraju se levi i desni podniz.
- Funkcija `merge_sorted_arrays` vrši spajanje sortiranih levih i desnih nizova u jedan sortirani niz.
- Proces spajanja se odvija upoređivanjem elemenata iz oba niza (`left_arr` i `right_arr`) i dodavanjem manjeg elementa u `sorted_arr`
- Nakon iscrpljivanja jednog od nizova, svi preostali elementi u drugom nizu se dodaju u `sorted_arr`
- Na kraju, funkcija `merge_sorted_arrays` vraća se sortirani niz

MergeSort - karakteristike

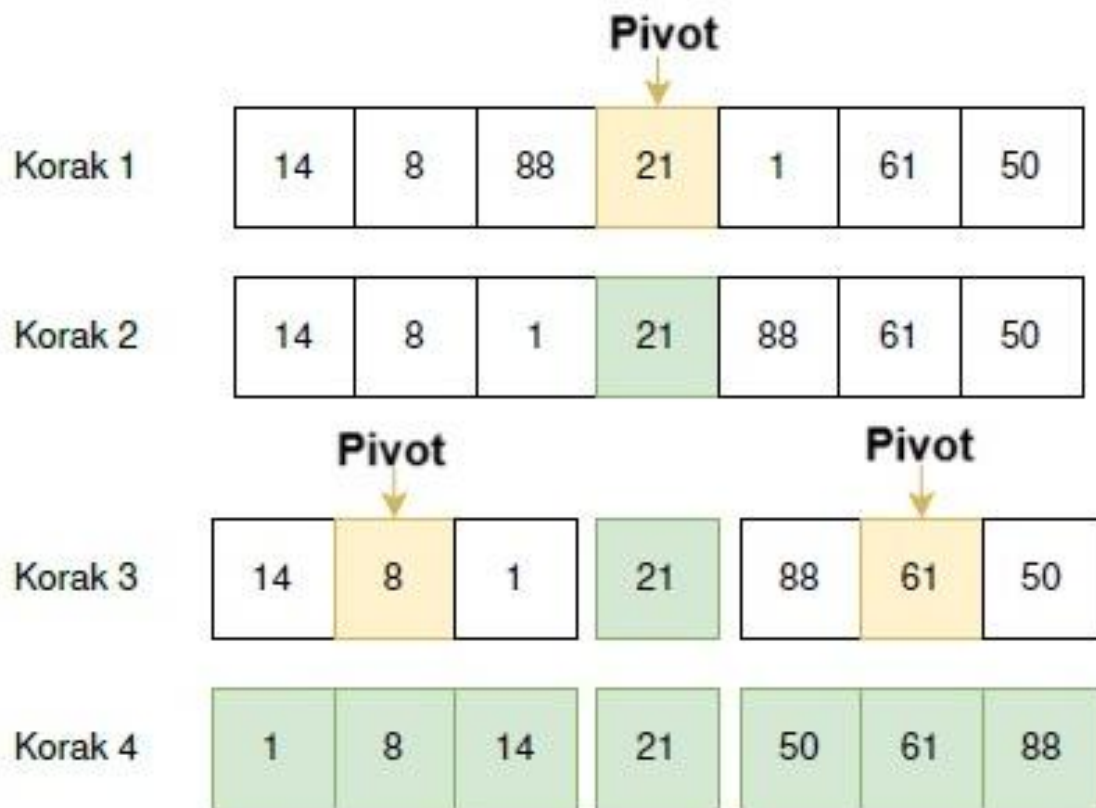
- Merge Sort je tipa „divide and conquer“ algoritma (kreator John von Neumann, 1945)
- Vremenska složenost algoritma je **$O(n \log_2 n)$**
- Faza spajanja podnizova ima složenost reda **$O(n)$** , što je i složenost svake faze poređenja elemenata niza. Pošto se u kompletnom postupku obavi **$k = \log_2 n$** ovakvih faza, može se zaključiti da red složenosti algoritma Merge Sort iznosi **$O(n) * O(\log_2 n) = O(n \log_2 n)$**
- Za algoritme ovog reda složenosti kaže se da se izvršavaju u **kvazilinearnom vremenu**
- Merge Sort je stabilan algoritam sortiranja
- Efikasan Quick Sort algoritam generalno prevazilazi Merge Sort algoritam
- Prednost metode Merge Sort, u odnosu na Selection Sort ogleda se u brzini izvršavanja prilikom velikog broja podataka, ali postoji nedostatak koji se odnosi na dodatnu memoriju koja je, za razliku od metode Selection Sort, u ovom slučaju neophodna
- Merge Sort je često najbolji izbor za sortiranje povezanih lista: relativno lako ga je implementirati jer zahteva samo **$O(1)$** dodatnog prostora

QuickSort algoritam sortiranja

QuickSort algoritam sortiranja

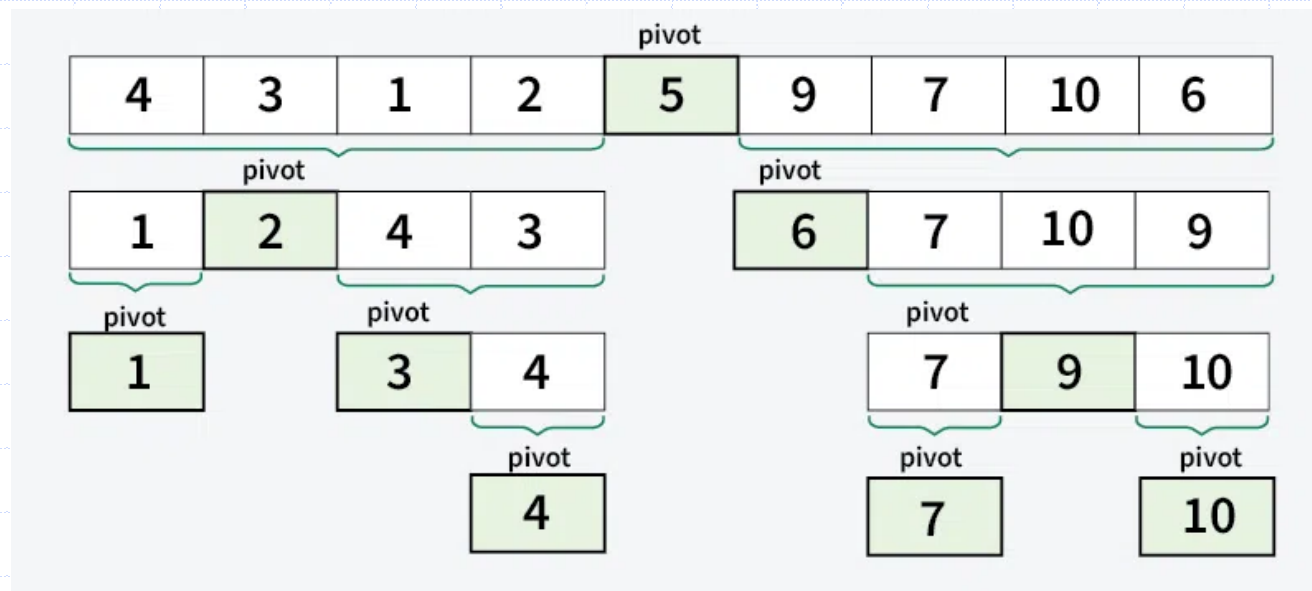
- **QuickSort** „brzo sortiranje“ jedan je od najbržih algoritama za sortiranje
- Koristi strategiju "podeli pa vladaj" pa je njegova prosečna vremenska složenost **$O(n \log_2 n)$**
- Osnovna ideja je da se u odnosu na odabrani **pivot element** niz sortira tako da svi brojevi manji od njega budu sa leve strane niza, a svi veći sa desne strane
- Zavisno od položaja pivot elementa, niz se rekurzivno deli na dva dela, tj. deo pre i deo posle pivot elementa, nakon čega se prethodno opisani postupak ponavlja za oba nova podniza
- QuickSort postoji u više verzija, koje se razlikuju u načinu izbora pivota, tako da pivot može biti odabran kao prvi, zadnji, srednji ili slučajni element niza
- Najčešće se bira da je pivot srednji ili početni element liste ili niza podataka

QuickSort



- Sortira se niz brojeva: **14, 8, 88, 21, 1, 61, 50**
- U početnom nizu od sedam elemenata bira se srednji element (broj 21) kao **pivot** element niza
- U 2. koraku sve brojeve manje od 21 prebacujemo levo, tj. pre pivot elementa, a sve veće na desnu stranu
- Kao rezultat dobijamo niz gde je broj 21 na ispravnom (sortiranom) mestu pa se može pristupiti podeli inicijalnog niza u dva dela
- U 3. koraku sada postoje dva podniza – jedan sa elementima levo od pivota i jedan sa elementima desno od pivota
- Na svakom od tih podnizova primenjujemo isti postupak, odnosno bismo srednji element niza kao pivot element, manje vrednosti smeštamo levo, a veće desno od pivot elementa
- Na kraju, u 4. koraku celi niz je sortiran

QuickSort



QuickSort algoritam se sastoji od sledeća 3 koraka:

- 1) Izbor pivota: pivot može biti prvi element, poslednji element, slučajni ili srednji element niza
- 2) Podela niza: niz se deli u odnosu na pivot, svi elementi manji od njega biće sa njegove leve strane, svi veći od njega biće sa njegove desne strane, pivot je tada na ispravnoj poziciji i dobijamo njegov indeks
- 3) Rekurzivni poziv: primenjuje se na oba particionisana podniza (levo i desno od pivota)

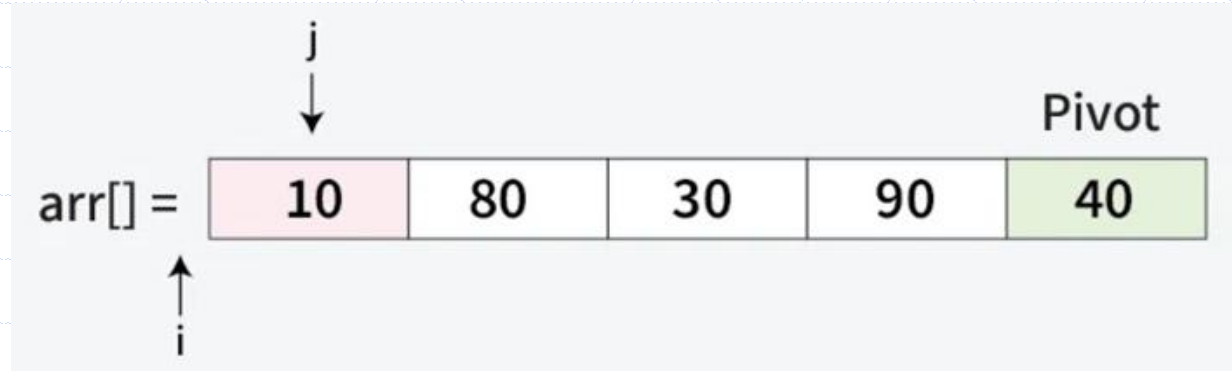
Osnovni slučaj: Rekurzija se zaustavlja kada u podnizu ostane samo jedan element, koji je podrazumevano sortiran

QuickSort

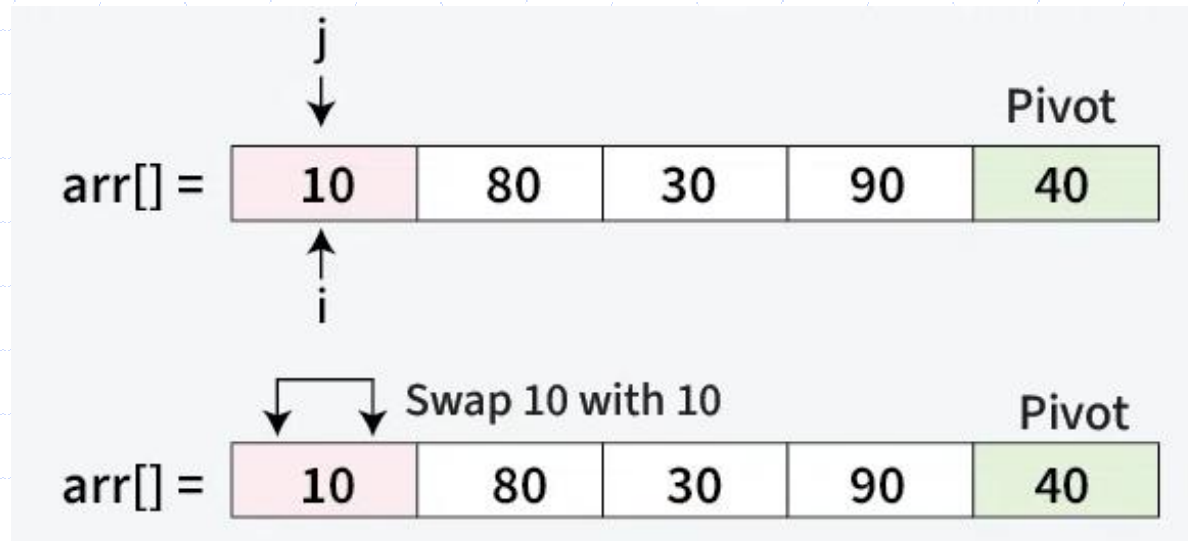
- Izbor pivota:
 - Prvi (ili poslednji) element kao pivot, problem sa ovim izborom je što se završava u najgorem slučaju kada je niz već sortiran
 - Slučajni element kao pivot, poželjan pristup jer se tada izbegava opcija za najgori slučaj
 - Srednji element kao pivot, najbolji pristup u smislu vremenske složenosti jer će funkcija particije uvek podeliti ulazni niz na dve polovine, ali u proseku je potrebno više vremena
- Particioni algoritam: Ključni proces u brzom sortiranju je metoda `partition()`, koriste se 3 algoritma i svi imaju vremensku složenost $O(n)$
 - **Naive Partition:** Kreira se kopija niza, prvo se postavljaju svi manji elementi, a zatim svi veći, a zatim se kopira privremeni niz nazad u originalni niz, što zahteva $O(n)$ dodatnog prostora
 - **Lomuto Partition:** Ovo je jednostavan algoritam, kod kojeg se prati indeks manjih elemenata i stalno se zamenjuju
 - **Hoare's Partition:** Najbrži algoritam, niz se obilazi sa obe strane i stalno se zamenjuje veći element sa leve strane sa manjim sa desne strane sve dok se niz ne podeli

QuickSort (Lomuto Partition)

- Korak 1:** Izbor pivota, bira se poslednji element **arr[4]=40** kao pivot, početni indeksi su **i=-1** i **j=0**

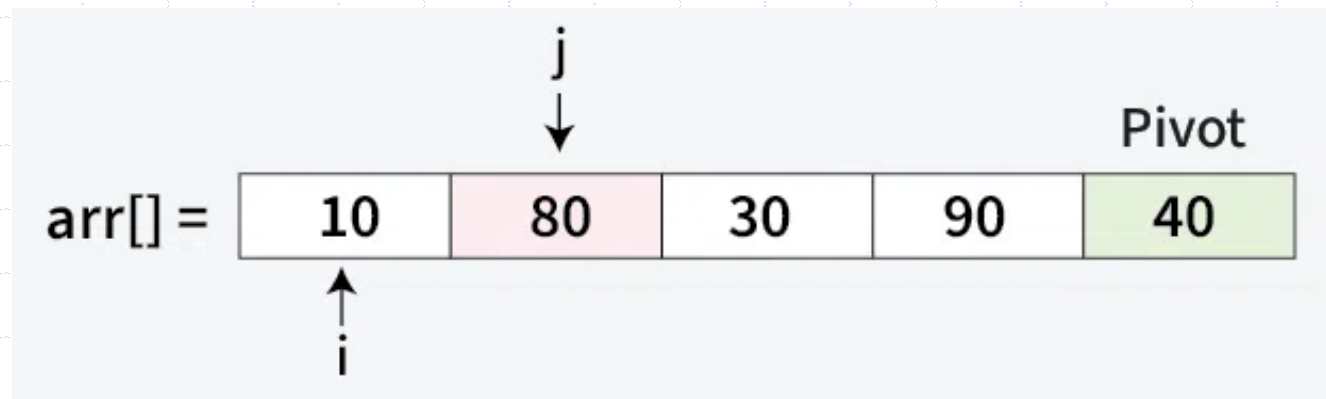


- Korak 2:** Pošto je **arr[j] < pivot** ili ($10 < 40$), **index i=0**, **arr[i]** i **arr[j]** menjaju mesta, **j=0**

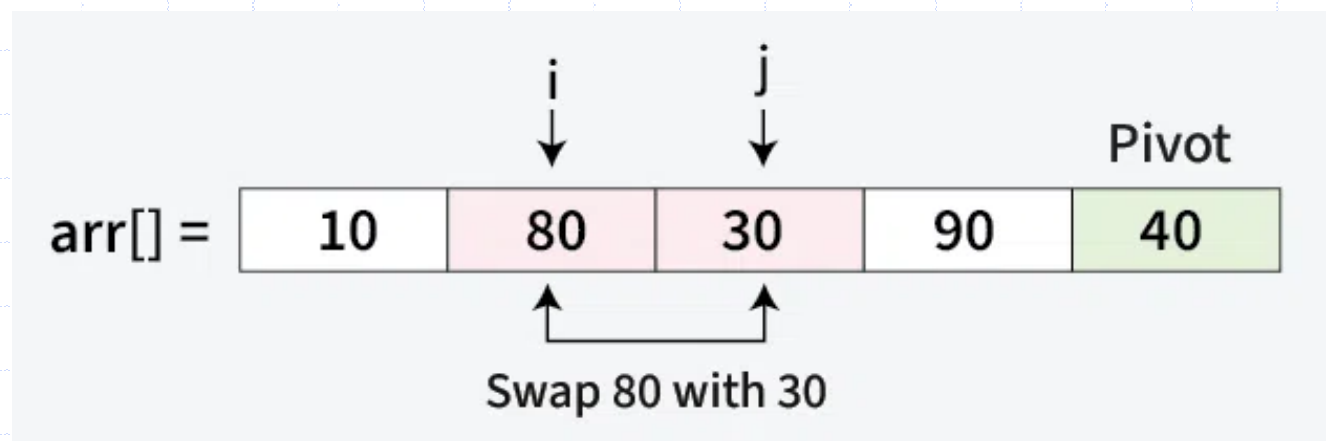


QuickSort (Lomuto Partition)

- **Korak 3:** Pošto je $\text{arr}[j] > \text{pivot}$ ($80 > 40$) nema zamene, index $j=1$

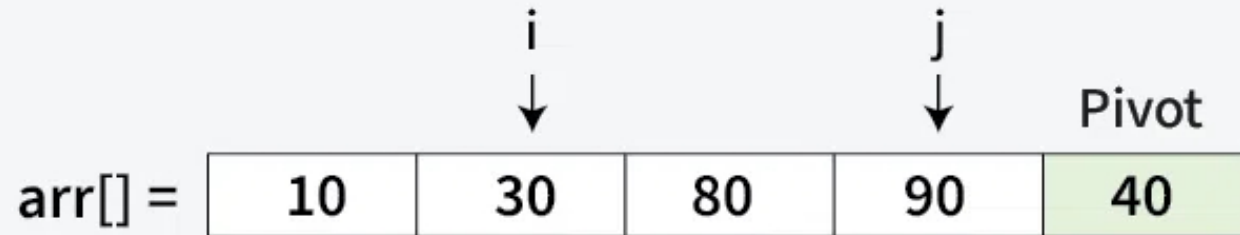


- **Korak 4:** Pošto je $\text{arr}[j] < \text{pivot}$ ($30 < 40$), index $i=1$, $\text{arr}[i]$ i $\text{arr}[j]$ menjaju mesta, $j=2$

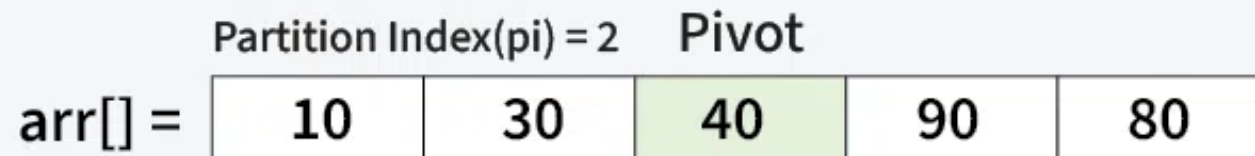
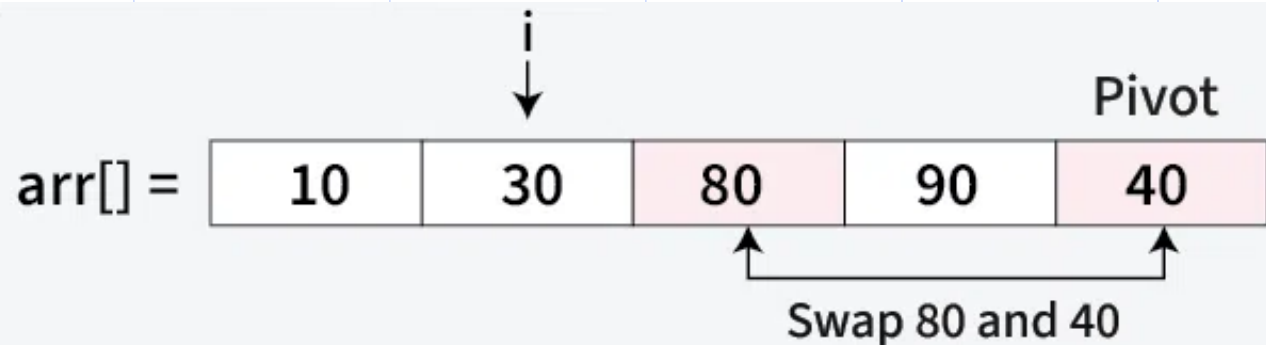


QuickSort (Lomuto Partition)

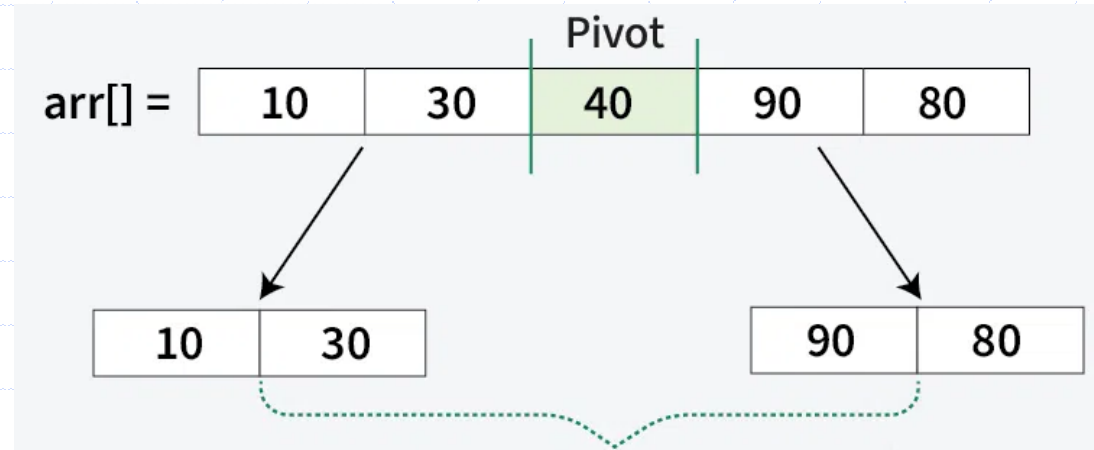
- **Korak 5:** Pošto je $\text{arr}[j] > \text{pivot}$ ($90 > 40$) nema zamene, index $j=3$



- **Korak 6:** Prolaz kroz niz je završen, pivot se postavlja na ispravnu poziciju, $\text{arr}[i+1] = \text{arr}[2]$ se zamenjuje sa $\text{arr}[4] = 40$



QuickSort (Lomuto Partition)



Poziv rekurzije QuickSort za levi podniz (od najmanjeg indexa do pivot-1) i za desni podniz (od pivot+1 do najvećeg indexa) kako bi se elementi sortirali levo i desno oko pivota

Proces particionisanja preuređuje niz na osnovu izabranog pivota, zatim se ista metoda rekurzivno primenjuje na manje podnizove levo i desno od pivota. Svaki put se biraju novi pivoti i ponovo se particioniraju nizovi. Ovaj proces se nastavlja sve dok ne ostane samo jedan element, koji je uvek sortiran. Kada se svaki element nađe na ispravnoj poziciji, ceo niz je sortiran.

QuickSort – programski kôd 1

```
1 def quick_sort(array):
2     quick_sort_helper(array, 0, len(array) - 1)
3     return array
4
5 def quick_sort_helper(array, start, end):
6     # Osnovni slučaj: podniz ima 0 ili 1 element
7     if start >= end:
8         return
9
10    # Izbor prvog elementa kao pivot
11    pivot = start
12    left = start + 1
13    right = end
14
15    while right >= left:
16        # Zamena elemenata ako su sa pogrešne strane pivotu...
17        if array[left] > array[pivot] and array[right] < array[pivot]:
18            array[left], array[right] = array[right], array[left]
19
20        if array[left] <= array[pivot]:
21            left += 1
22
23        if array[right] >= array[pivot]:
24            right -= 1
25
```

```
26    # Premestiti pivot na njegovu sortiranu poziciju
27    array[pivot], array[right] = array[right], array[pivot]
28
29    # Rekurzivno sortiranje podnizova
30    is_left_subarray_smaller = right - 1 - start < end - (right + 1)
31
32    if is_left_subarray_smaller:
33        quick_sort_helper(array, start, right - 1)
34        quick_sort_helper(array, right + 1, end)
35    else:
36        quick_sort_helper(array, right + 1, end)
37        quick_sort_helper(array, start, right - 1)
38
39    arr = [134, 95, 74, 55, 32, 11, 8, 2]
40    print(f"Ulazni niz: arr = {arr}")
41    print("Sortirani niz:", quick_sort(arr)) # Output [1, 3, 5, 7, 9]
42
```



- Ulazni niz: arr = [134, 95, 74, 55, 32, 11, 8, 2]
Sortirani niz: [2, 8, 11, 32, 55, 74, 95, 134]

QuickSort

Kao i Merge Sort, **QuickSort** je algoritam sortiranja po principu „podeli pa vladaj“ kojim se prvo bira element pivot, a zatim particioniše niz oko njega. Zatim se vrši rekurzivno sortiranje podnizove sa obe strane pivot, dok se ceo niz ne sortira.

- Funkcija **quick_sort** poziva funkciju **quick_sort_helper** kako bi se izvršilo sortiranje
- Funkcija **quick_sort_helper** uzima početni i krajnji indeks podniza koji treba sortirati
- U svakom rekurzivnom pozivu, funkcija bira pivot element (u ovom slučaju, prvi element) i particioniše niz oko pivot.
- Proces particionisanja se vrši pomoću dva pokazivača, levog i desnog, koji se kreću jedan prema drugom dok se ne ukrste
- Elementi se upoređuju sa pivotom, i ako su na pogrešnoj strani, zamenjuju se
- Nakon procesa particionisanja, pivot element je u svojoj sortiranoj poziciji, a podniz je podeljen na dva dela.
- U zavisnosti od veličine podniza, funkcija vrši rekurzivne pozive da bi prvo sortirala manji podniz kako bi se minimizirao prostor na steku koji se koristi za rekurziju.
- Proces se nastavlja dok se ne dostigne osnovni slučaj (podniz sa 0 ili 1 elementom)

QuickSort algoritam ima prosečnu vremensku složenost od **$O(n \log(n))$** , gde je **n** veličina ulaznog niza. Za najgori slučaj vremenska složenost može biti **$O(n^2)$** kada je izbor pivot nepovoljan. Korak particionisanja ima linearnu složenost, a rekurzivni pozivi se vrše na manjim podnizovima.

QuickSort - karakteristike

- Algoritam QuickSort spada u najefikasnije postupke za sortiranje u najvećem broju praktičnih situacija
- Njegova vremenska složenost u prosečnom slučaju iznosi **$O(n \log_2 n)$** pa je značajno efikasniji od Merge Sort algoritma
- U najnepovoljnijem slučaju njegov red složenosti iznosi **$O(n^2)$** što je lošije od Merge Sorta
- Prednost QuickSorta u odnosu na Merge sort je da ne koristi dodatnu memoriju (In-place algoritam sortiranja)
- Najgori slučaj zavisi od vrednosti izabranog pivota, a u slučaju da je to središnji element, jedan od mogućih najnepovoljnijih slučajeva ne odnosi se na opadajuću listu, već na listu u kojoj su svi elementi isti. Tada pri svakom pozivu funkcije `podeli()` svi parovi elemenata s početka i kraja zamenjuju mesta, uz vraćanje središnjeg indeksa kao kraja levog podniza i tada se u svim rekurzivnim pozivima obavlja maksimalan broj zamena
- Značajno se koristi u programskim jezicima:
 - Primitivni tipovi – QuickSort se uglavnom koristi za brzo sortiranje
 - Referentni tipovi/objekti – Merge Sort se najčešće koristi za sortiranje

RadixSort algoritam sortiranja

Sortiranje razvrstavanjem (RadixSort)

- MergeSort, HeapSort, QuickSort algoritmi sortiranja imaju logaritamsku vremensku složenost **$O(n \log n)$**
- Svaki ovaj algoritam se zasniva na različitim načinima međusobnog poređenja elemenata niza ili liste i za najgori slučaj potrebno je napraviti **$O(n \log n)$** poređenja da bi se sortirala sekvenca od **n** elemenata
- Da li je moguće postići efikasniji način sortiranja?
- Uz neke prepostavke o podacima i upotrebom podataka na specifičan način može se postići linearna složenost algoritma sortiranja
- Jedno od rešenja je „sortiranje višestrukim razvrstavanjem“ ili **Radix Sort**
- Sortiranje višestrukim razvrstavanjem predstavlja prirodno uopštenje najjednostavnijeg postupka sortiranja koji se sastoji u osiguravanju dovoljnog broja lokacija kako bi se svaki element smestio na svoju lokaciju
- Radix sortiranje koristi pozicionu reprezentaciju broja (ili niza karaktera) i odvojeno analizira cifre (ili znakove) na raznim pozicijama

RadixSort

```
RADIX-SORT( $A, d$ )  
1 for  $i \leftarrow 1$  to  $d$   
2 do use a stable sort to sort array  $A$  on digit  $i$ 
```

- Ulazni niz ili lista se sastoje od n ulaznih elemenata koji se sastoje od celih brojeva (digits) sa k cifara $d_{k-1}, d_{k-2}, \dots, d_0$
- U prvom koraku vrši se sortiranje po najmlađoj cifri (cifra jedinica). Uzimaju se redom elementi iz neuređenog niza i razvrstavaju u deset redova $Q_0 \dots Q_9$ po vrednosti najmlađe cifre, tako što se tekući element stavlja na kraj odgovarajućeg reda
- Kad se obrade svi elementi, svi redovi se spoje u jedan niz po poretku $Q_0, Q_1 \dots Q_9$
- U sledećem koraku se opet uzimaju elementi ovog niza redom i razvrstavaju u redove, ali po vrednosti druge cifre (desetice) d_1 , pa se opet na kraju svi redovi spoje
- Postupak se završava kad se izvrši uređenje i po najstarijoj cifri d_{k-1}

RadixSort

RadixSort algoritam za sortiranje niza [170, 45, 75, 90, 802, 24, 2, 66] podrazumeva korake:

- **Korak 1:** Pronaći najveći element, broj **802** koji ima 3 cifre pa će se vršiti iteracija 3 puta
- **Korak 2:** Prva iteracija, ortiranje elemenata na osnovu vrednosti **cifara jedinica** ($X=0$), pri čemu se najveće vrednosti pozicioniraju u desnu stranu niza...



RadixSort

RadixSort algoritam za sortiranje niza [170, 45, 75, 90, 802, 24, 2, 66] podrazumeva korake:

- **Korak 3:** Druga iteracija, sortiranje elemenata na osnovu vrednosti **cifara desetica** ($X=1$), pri čemu se najveće vrednosti pozicioniraju u desnu stranu niza...



RadixSort

RadixSort algoritam za sortiranje niza [170, 45, 75, 90, 802, 24, 2, 66] podrazumeva korake:

- **Korak 4:** Treća iteracija, sortiranje elemenata na osnovu vrednosti **cifara stotina** ($X=2$), pri čemu se najveće vrednosti opet pozicioniraju u desnu stranu niza...



- **Sortiran niz:**



RadixSort - primer

- **PRIMER:** Niz veličine $k=6$, $d=5$

A	33101	26440	16341	2332	20101	801
----------	--------------	--------------	--------------	-------------	--------------	------------

- ▶ Raspored cifara jedinica: 1 0 1 2 1 1 → Sortiranje prebrojavanjem: 0 1 1 1 1 2

A	26440	33101	16341	20101	801	2332
----------	--------------	--------------	--------------	--------------	------------	-------------

- ▶ Raspored cifara desetica: 4 0 4 0 0 3 → Sortiranje prebrojavanjem: 0 0 0 3 4 4

A	33101	20101	801	2332	26440	16341
----------	--------------	--------------	------------	-------------	--------------	--------------

- ▶ Raspored cifara stotina: 1 1 8 3 4 3 → Sortiranje prebrojavanjem: 1 1 3 3 4 8

RadixSort - primer

- **PRIMER:** Niz veličine $k=6$, $d=5$

A	33101	20101	2332	16341	26440	801
----------	--------------	--------------	-------------	--------------	--------------	------------

- ▶ Raspored cifara hiljada: 3 0 2 6 6 0 → Sortiranje prebrojavanjem: 0 0 2 3 6 6

A	20101	801	2332	33101	16341	26440
----------	--------------	------------	-------------	--------------	--------------	--------------

- ▶ Raspored cifara deset hiljada: 2 0 0 3 1 2 → Sortiranje prebrojavanjem: 0 0 1 2 2 3
- ▶ **Konačno sortiran niz:**

A	801	2332	16341	20101	26440	33101
----------	------------	-------------	--------------	--------------	--------------	--------------

RadixSort

- Suština RadixSort algoritma je u stabilnosti procesa sortiranja, koja je omogućena ubacivanjem elementa u red na njegov kraj
- Kada se u i -tom koraku vrši sortiranje po cifri d_{i-1} , veći je onaj element koji ima višu cifru na toj poziciji i on ide u odgovarajući red, elementi sa istom cifrom idu u isti red, ali su oni zbog stabilnosti metoda već uređeni po nižim ciframa, što omogućava spajanje redova bez vođenja računa o njihovim granicama
- Prilikom implementacije algoritma treba voditi računa o tome da se broj elemenata u redovima u pojedinim koracima ne može predvideti, pa zato sekvencijalna realizacija redova nije pogodna, već treba pribeći ulančanoj reprezentaciji
- Redovi se održavaju kao ulančane liste u koje se novi elementi stavljaju na kraj i zbog toga za svaki red treba obezbediti pokazivače koji pokazuju na početak i kraj liste
- Na početku se od neuređenog niza napravi jedna lista iz koje se uzimaju elementi i stavljaju u redove. Posle svakog koraka sve liste se opet nadovezuju u jednu listu, počevši od liste koja odgovara cifri 0 do liste koja odgovara cifri 9

RadixSort - programski kôd 1

```
1 def radix_sort(array):
2     if len(array) == 0:
3         return array
4     max_number = max(array)
5     num_digits = len(str(max_number))
6     for digit in range(num_digits):
7         counting_sort(array, digit)
8     return array
9
10 def counting_sort(array, digit):
11     sorted_array = [0] * len(array)
12     count_array = [0] * 10
13
14     for num in array:
15         count_index = (num // (10 ** digit)) % 10
16         count_array[count_index] += 1
17
18     for idx in range(1, 10):
19         count_array[idx] += count_array[idx - 1]
20
21     for idx in range(len(array) - 1, -1, -1):
22         count_index = (array[idx] // (10 ** digit)) % 10
23         count_array[count_index] -= 1
24         sorted_index = count_array[count_index]
25         sorted_array[sorted_index] = array[idx]
26
27     for idx in range(len(array)):
28         array[idx] = sorted_array[idx]
29
30 array = [321, 56, 789, 12, 456, 234, 901]
31 print(f"Ulazni niz: {array}")
32 radix_sort(array)
33 print("Sortirani niz:", array)
```



● Ulazni niz: [321, 56, 789, 12, 456, 234, 901]
Sortirani niz: [12, 56, 234, 321, 456, 789, 901]

RadixSort je algoritam sortiranja koji ne koristi poređenje elemenata niza, već procesiranje cifara elemenata. Počinje sortiranjem brojeva na osnovu cifre jedinica i postepeno se kreće ka najvećoj klasi cifre zapisa elementa.

- Funkcija `radix_sort` je početna tačka algoritma, proverava da li je ulazni niz prazan i pronalazi maksimalni broj, kao i broj njegovih cifara. Zatim se vrši radix sortiranje pozivanjem funkcije `counting_sort` za svaku poziciju cifre

- Funkcija `counting_sort` se koristi kao potprogram za sortiranje brojeva na osnovu trenutne cifre. Ona kreira niz `count_array` za brojanje pojavljivanja svake vrednosti cifre (0 do 9) na trenutnoj poziciji cifre, to je sortiranje prebrojavanjem

- Niz `count_array` je modifikovan kako bi se sačuvale pozicije gde svaka vrednost cifre treba da se postavi u sortiranom nizu

- Sortirani niz se konstruiše iteracijom kroz ulazni niz obrnutim redosledom i postavljanjem svakog elementa na ispravnu poziciju na osnovu `count_array`

- Na kraju, originalni niz se ažurira sortiranim elementima

RadixSort

- Vremenska složenost RadixSort algoritma zavisi od broja cifara k i od broja ulaznih elemenata n
- Pošto se u svakom od k koraka obrađuju svih n elemenata, vremenska složenost ovog algoritma je $O(kn)$
- Ako je broj cifara k konstantan, onda je složenost **linearna**
- Ovo je efikasan metod za nizove sa elementima manjeg broja cifara k , za elemente sa većim brojem cifara performanse algoritma opadaju, pa se tada usvaja neko od hibridnih rešenja
- Princip radix sortiranja se može primeniti samo na manji broj najstarijih cifara, što neće potpuno sortirati niz, ali će ga dosta dobro urediti, a završno sortiranje se može obaviti nekom metodom za uređene nizove, kao npr. direktno umetanje
- Ako k nije konstanta već raste sa n , onda se povećava i vremenska složenost, a ako k pokriva skoro čitav skup mogućih vrednosti, onda se približava ka $\log n$, a složenost se približava ka $O(n \log n)$

